

Relatório BII

Nóis

6 de agosto de 2026

1 Introdução

A lista ligada, também conhecida por *cons list*, é um pilar da programação funcional e uma das estruturas de dados mais simples e elegantes. Como as listas ligadas são apenas células ligadas sequencialmente por ponteiros, operações como *cons* (adicionar um elemento ao início), *car* (aceder ao primeiro elemento) e *cdr* (aceder à lista excluindo o primeiro elemento) apenas requerem tempo e espaço $O(1)$. No entanto, aceder a uma célula arbitrária da lista exige percorrer até $n - 1$ ponteiros para uma lista de comprimento n .

Para resolver este problema, Eugene W. Myers [1] propôs uma nova estrutura de dados, a *Applicative Random-Access Stack*, que denominamos *Myers List*. Essa estrutura mantém as operações de *cons*, *car* e *cdr* em tempo constante, permitindo ainda reduzir a complexidade de *lookup* para uma constante de $O(\log n)$, para ser mais preciso $3 \log(n) - 5$ pointers. Esta melhoria é possível observar pela adição de um ponteiro extra, denominado *jump*, em cada célula.

Trabalhos mais recentes, como o de Edward Peters, Yong Qi Foo e Michael D. Adams [2], aprimoraram a construção e o uso desse ponteiro *jump*, resultando nas variantes apresentadas no paper, *Improved Myers List* e *Advanced Myers List*. Essas estruturas reduziram ainda mais o número de travessias necessárias no pior caso, obtendo uma complexidade de $2 \log(n + 1) - 3$ e $(1 + 1/s) \log(n) + s + 9$ respetivamente, aproximando-se do limite teórico de $\log(n + 1) - 1$ (teoria da informação).

Apesar dessas melhorias, a *Advanced Myers List* apresenta uma limitação, a perda da funcionalidade de *cons* em tempo constante, funcionalidade que acreditamos ser de extrema importância. Além disso, durante todo o estudo do paper referenciado, os autores propõem algumas restrições, sendo uma delas a não adição de novos apontadores em cada célula. Isso motiva as duas perguntas centrais deste trabalho: **(i)** é possível recuperar a operação *cons* em tempo constante na *Advanced Myers List*, removendo a restrição do projeto, ou seja, permitindo a adição de ponteiros extras por célula enquanto se preserva a lógica e a estrutura apresentada no paper? **(ii)** removendo novamente a mesma restrição, é possível aplicar as ideias centrais propostas para a *Advanced Myers List* à *Improved Myers List*, de modo a melhorar o seu desempenho de *lookup* e mantendo o *cons* constante?

Neste relatório, propomos duas novas estruturas de dados e as suas respetivas implementações e bibliotecas, que respondem a estas duas perguntas: a *Dense Advanced Myers List* (Secção 3.1) e a *Improved Applicative Random-Access Stack with Sigma region* (*IARASS*) (Secção 3.2). Na secção 4 apresentamos os resultados experimentais de desempenho e de uso de memória. Na secção 5 discutimos as implementações destes resultados face às perguntas colocadas.

2 Enquadramento Teórico

2.1 Basic Myers List

A Myers List original, proposta por Eugene W. Myers (1983), introduz um apontador jump adicional em cada célula de uma cons list, permitindo lookup em $\lceil 3 \log(n) - 5 \rceil$ apontadores no pior caso. A sua construção iterativa preserva o suporte a cons em tempo $O(1)$, mas o seu desempenho assintótico fica aquém do limite teórico de informação.

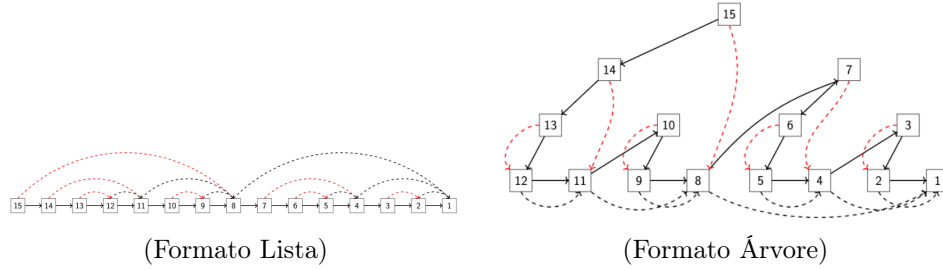


Figura 1: Representação da Basic Myers List

A associação do ponteiro jump funciona da seguinte forma:

$$c.\text{jump} = \begin{cases} \text{zs} & \text{if } xs.\text{length} - ys.\text{length} = ys.\text{length} - zs.\text{length} \\ xs & \text{otherwise} \end{cases}$$

Figura 2: Jump

Lookup na Basic Myers List é bastante simples, assume-se que len é um valor válido:

$$\text{lookup } xs \text{ len} = \begin{cases} xs & \text{if } xs.\text{length} = \text{len} \\ \text{lookup } xs.\text{jump} \text{ len} & \text{if } xs.\text{jump}.\text{length} \geq \text{len} \\ \text{lookup } xs.\text{next} \text{ len} & \text{otherwise} \end{cases}$$

Figura 3: Lookup

2.2 Improved Myers List

A Improved Myers List, proposta por Peters, Foo e Adams (2025), introduz uma modificação simples, mas eficaz à maneira como a Basic Myers List é construída. O apontador jump da primeira célula de cada região passa a apontar para a primeira célula da segunda sub-região, em vez da última célula. Assim, o resultado é uma melhoria do pior caso de lookup de $3(\log n) - 5$ para $2(\log(n + 1)) - 3$ apontadores, igualando ao desempenho das Okasaki List, mas ainda com a vantagem de que permite o lookup a partir de qualquer posição.

2.2.1 Construção e conceito de região

Uma região é definida pelo seu grau k :

- Uma região de grau $k=0$ consiste numa única célula.
- Uma região de grau $k>0$ é composta por duas sub-regiões de grau $k-1$ (no paper, estas regiões são denominadas Q_{k-1} e R_{k-1}) e uma célula adicional no início.

Na Basic Myers List, o apontador jump da célula inicial de uma região aponta para a última célula dessa mesma região. Adicionalmente, as duas sub-regiões partilham células, o que torna a estrutura menos eficiente. A Improved Myers List resolve este problema alterando a regra de construção: as sub-regiões Q_{k-1} e R_{k-1} deixam de partilhar células, e o apontador jump da célula inicial passa a apontar diretamente para a primeira célula de R_{k-1} . Se visualizarmos a lista como uma árvore, as sub-regiões Q_{k-1} e R_{k-1} formam, respetivamente, as sub-árvores esquerda e direita. Assim, o acesso à raiz da sub-árvore direita passa a custar apenas a travessia de um único apontador (jump), eliminando a ineficiência da variante clássica.

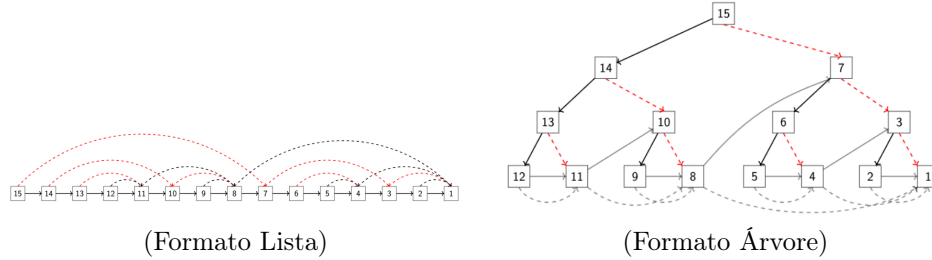


Figura 4: Representação da Improved Myers List

2.2.2 Lookup

O algoritmo de lookup na Improved Myers List é idêntico ao da Basic Myers List, percorre jump se $xs.jump.length \geq len$, caso contrário, percorre next.

2.3 Advanced Myers List

A Advanced Myers List, introduzida por Peters et al. (2025), parte da Improved Myers List e introduz apontadores adicionais (uncle, inc, greater), permitindo assim um lookup que pode ser realizado em tempo aproximadamente logarítmico $((1+1/\sigma) \lceil \log n \rceil + 9 + \sigma)$ à custa de maior complexidade estrutural e de construção.

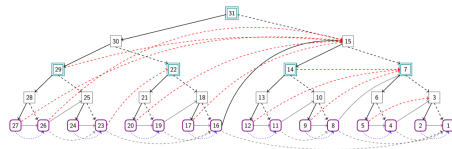


Figura 5: Advanced Myers List

2.3.1 Tipos de Nós

A Advanced Myers List organiza as células em o que é chamado de nós lógicos, estes consistem em uma célula sozinha ou a junção de duas ou três células. Existem três tipos de nós:

- Nós normais - são constituídos por apenas uma única célula (top), contendo apontadores next e jump a apontar diretamente para os filhos esquerdo e direito.
- Nós skip - são constituídos por duas células, denominadas top e bottom. O apontador next da célula top liga diretamente à célula bottom e o apontador jump aponta para o uncle. Estes nós aparecem repetidamente em camadas específicas, mais concretamente a cada intervalo acima da camada folha.
- Nós folha - São constituídos pela junção de três células: top, middle e bottom. Os apontadores jump de cada célula são reaproveitados para atalhos distintos: inc, greater e uncle. Estes nós constituem inteiramente a camada inferior da árvore lógica.

2.3.2 Construção

Antes de discutir a construção é importante primeiro compreender os seguintes conceitos:

- Right-Handed Depth (RHD) - Corresponde à distância do ancestral mais próximo que é um filho esquerdo.
- Right-Edge Depth (RED) - Corresponde à distância ao ancestral mais próximo que se encontra no caminho mais à direita a partir da raiz da árvore.

A construção da Advanced Myers List perde a função de cons em tempo constante, pois exige saber previamente o número exato de elementos para calcular a altura da árvore lógica e formar uma árvore binária perfeita, utilizando elementos que servem como fillers caso necessário. A estrutura é construída recursivamente do head até a camada de folhas e a atribuição dos apontadores depende do tipo de nó:

- Nós normais e skip: O apontador next aponta para a sub-árvore esquerda e o jump para a sub-árvore direita
- Nós skip e folhas: O apontador uncle liga a um tio onde a regra $\text{RED}(s.\text{uncle}) = \text{RHD}(s)$ é verificada
- Nós folha: O apontador inc liga a uma folha mais próxima à direita tal que $\text{RHD}(f') = \text{RHD}(f) + 1$. O apontador greater lia à folha mais próxima à direita onde $\text{RHD}(f') > \text{RHD}(f)$

3 Estruturas Propostas

3.1 Dense Advanced Myers List

A Dense Advanced trata-se de uma variação da Advanced Myers List criada por Peters et al. (2025), mas em vez de dividir um nó com dois a quatro ponteiros, em um a três células, cada uma com dois ponteiros. Decidimos representar cada nó com uma única célula, com o seu respetivo número de ponteiros:

```
type 'a add_points =
  | Normal
  | Skip of 'a cell
  | Leaf of 'a cell * 'a cell
and
'a cell = { next : 'a cell; jump : 'a cell; length : int;
  value : 'a; rhd : int; red : int; more : 'a add_points }
```

Listing 1: Tipos de células

Isto tornou a implementação da estrutura mais fácil, e por consequência, determinar o algoritmo para a expressão cons foi mais simples. Outros benefícios desta alteração foram: Um número de travessias de ponteiros menor; Encontraram-se novas otimizações que não foram ponderadas na implementação original; Obteve-se um lookup mais rápido.

No entanto, um defeito desta estrutura em relação à original, é que esta gasta mais memória, já que cada célula contém mais ponteiros.

O caso uso mais apropriado para esta estrutura seria um ambiente onde a travessia de ponteiros seria muito custosa, tal como referido para a lista original: «Thus, advanced Myers lists may not be appropriate for in-memory lists, but they may be useful for applications where the cost of pointer traversals is high relative to other computations, such as when pointer traversals involve network access.» (Peters et al.,2025)

3.2 Improved Applicative Random-Access Stack with Sigma Region

A Improved Applicative Random-Access Stack with Sigma region (IARASS) é uma variante da Improved Myers List que se beneficia da introdução do con-

ceito de sigma layers, introduzido por Peters et al. (2025) na Advanced Myers List. Assim, o objetivo da IARASS é utilizar este novo apontador para aliviar o problema da dupla descida (double descent) identificado no paper, sem perder a simplicidade estrutural existente na variante Improved. Na secção dos Resultados, mostraremos ainda que esta estrutura, como é de esperado, apresenta um maior uso de memória que a Improved Myers List, no entanto apresenta um menor uso de memória quando comparada com a Advanced Myers List e apresentando um maior desempenho no número de travessias quando comparada com ambas as variantes.

Ao contrário da Improved Myers List em que todas as células apresentam a mesma estrutura, contendo os campos next, jump, length e value, a IARASS distingue dois tipos de células:

```
type 'a cell =
| Normal of {next:'a cell; jump:'a cell; length:int;
             value:'a; height:int }
| Skip of {next:'a cell; jump:'a cell; length:int;
           value:'a; height:int; shortcut:'a cell }
```

Listing 2: Tipos de células

Nesta variante, as células normais mantêm apenas os dois apontadores já existente na variante Improved Myers List mantendo toda a lógica já existente, enquanto as células skip (células de salto) apresentam um terceiro apontador, denominado shortcut. A decisão de qual célula é criada é baseada na altura (height), sempre que a altura é um divisor de sigma, ou seja $\text{height} \bmod \text{sigma} = 0$, a célula criada é do tipo skip e nas restantes camadas serão do tipo normal. Ao contrario da Advanced Myers List, um sigma maior não indica uma melhor performance no lookup, nesta estrutura a lógica é inversa, ou seja, quanto maior o sigma maior será o número de travessias, mas consequentemente usará menos memória.

O novo apontador shortcut destas células skip é a contribuição principal para resolver o problema da dupla descida e assume um comportamento diferente dependendo da posição da célula na árvore lógica: **(i)** quando a altura é diferente de 0, o shortcut aponta para a última célula da região que contém a célula acabada de ser criada. **(ii)** quando a altura é igual a zero, o shortcut aponta para c.jump.jump, permitindo uma travessia horizontal mais rápida do que antes.

4 Resultados

Nesta secção, apresentaremos os resultados obtidos na avaliação das cinco estruturas abordadas neste relatório, Basic, Improved e Advanced Myers List, Dense Advanced Myers List e IARASS.

4.1 Número de Travessias

Os gráficos das Tabelas 1 e 2 comparam o número de travessias do pior caso com o limite teórico da estrutura, para listas de comprimento entre 1 e 1_000_000 elementos.

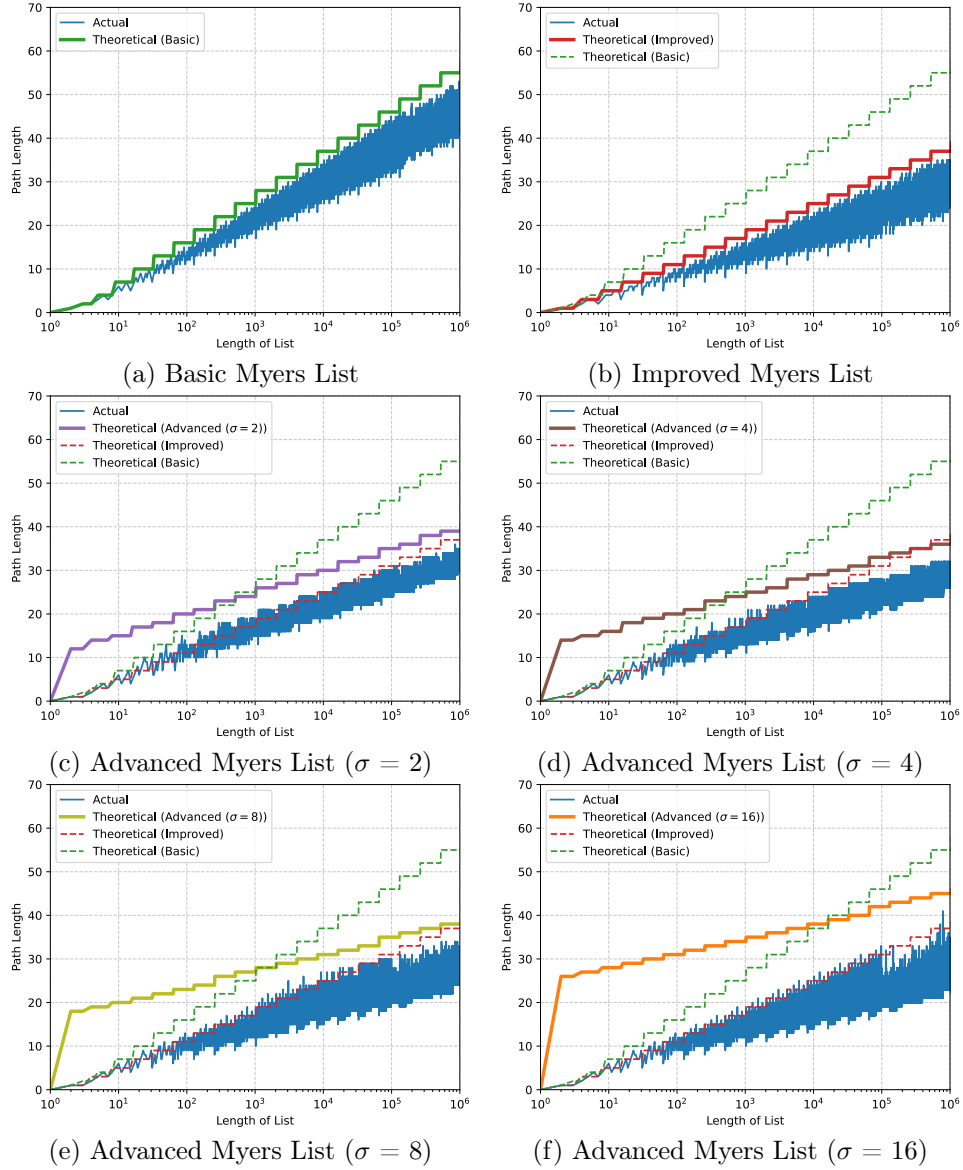
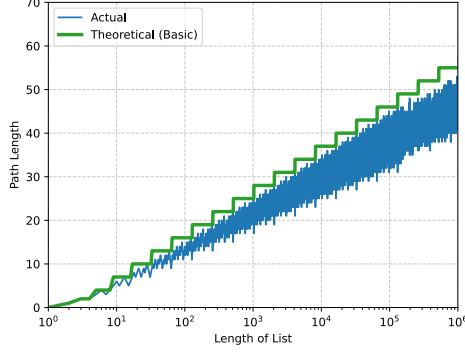


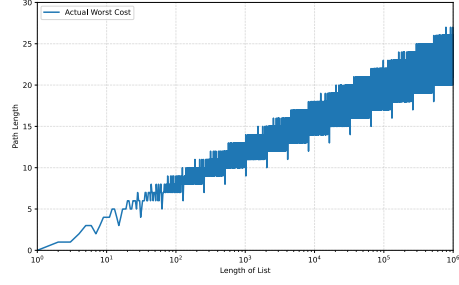
Tabela 1: Número de travessias - Basic, Improved e Advanced Myers List

Os resultados confirmam a hierarquia Basic > Improved >= Advanced, mas

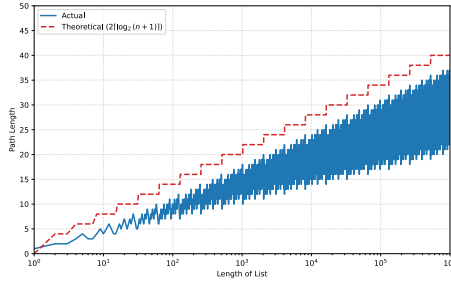
mostram também que a escolha de um sigma tem um impacto direto no ponto a partir do qual a Advanced Myers List compensa mais face à Improved Myers List.



(a) Dense Advanced Myers List



(b) Iarass



(c) Okasaki List

Tabela 2: Número de travessias - Dense Advanced Myers List, IARASS e Okasaki List

A Tabela 3 apresenta o número máximo de travessias observado para $n = 1\,000\,000$, valores retirados da leitura dos gráficos anteriores com valores exatos para comparação direta entre estruturas.

Variante	Pior caso registado
Basic Myers List	53
Improved Myers List	35
Advanced Myers List	[32, 41] (depende do sigma)
Dense Advanced Myers List (sigma = 2)	24
Iarass	27
Okasaki List	37

Tabela 3: Pior caso de travessias registado ($n = 1\,000\,000$)

4.2 Tempo de Execução

A Tabela 4 apresenta o tempo total de execução de lookups sobre todos os elementos de uma lista com 1 000 000 de elementos para todas as variantes.

Variante	Lookup All (1_000_000)
Basic Myers List	57.42 ms
Improved Myers List	77.46 ms
Advanced Myers List	401.43 ms
Dense Advanced Myers List ($\sigma = 2$)	60.35 ms
Iarass	65.00 ms
Okasaki List	51.04 ms

Tabela 4: Tempo de execução

4.3 Uso de Memória

A Tabela 5 apresenta o uso de memória de cada variante para uma lista de 1 000 000 de elementos. Como era esperado, a Basic e a Improved Myers List apresentam exatamente o mesmo uso de memória (38.15 MB), uma vez que ambas mantêm o mesmo número de apontadores por célula, sendo apenas diferentes na maneira como o apontador jump é atribuído durante o cons. A Okasaki List apresenta o menor uso de memória de todas as variantes (22.89 MB), por não incluir nenhum apontador adicional.

A Advanced Myers List (53.41 MB) apresenta um uso de memória superior ao das variantes Basic e Improved não por introduzir novos apontadores, a estrutura de célula mantém-se igual, reaproveitando o próprio apontador jump para servir de uncle/inc/greater consoante o tipo de nó lógico, mas por cada célula ter de guardar explicitamente a informação de RHD e RED necessária para a sua construção e para a otimização do lookup face as variantes anteriores.

A Dense Advanced Myers List (75.02 MB) leva esta ideia mais longe, ao consolidar cada nó lógico numa única célula, os campos rhd, red e o identificador do tipo de nó (o campo more, com os construtores Normal, Skip e Leaf) passam a estar presentes em todas as células, que consequentemente exige um consumo de memória ainda superior ao da Advanced original.

Variante	Memória (1000000 elementos)
Basic Myers List	38.15 MB
Improved Myers List	38.15 MB
Advanced Myers List	53.41 MB
Dense Advanced Myers List ($\sigma = 2$)	75.02 MB
Iarass	50.86 MB
Okasaki List	22.89 MB

Tabela 5: Uso de memória para cada variante

5 Discussão

Respostas às perguntas:

Relativamente à primeira pergunta, a implementação da Dense Advanced Myers List concluiu que não foi possível obter uma operação cons em tempo constante. Esta limitação deve-se à impossibilidade de aceder a uma determinada célula, que será utilizada como ponteiro uncle, greater ou inc. No entanto, foi possível implementar uma operação cons funcional para esta estrutura, com uma complexidade temporal de (... TODO), (Referir que a tentativa de implementar em cons $O(1)$ pode ficar para futuras investigações? ou isso mostra que falhamos? XD, tipo dizer que para existir um cons $O(1)$ será necessário repensar a estrutura usada ...).

Apesar de o cons em $O(1)$ não ter sido obtido, a Dense Advanced Myers List apresentou resultados eficazes na leitura, tendo um tempo de execução de

lookup total de 60.35 ms, valor muito inferior à variante Advanced Myers List com 401.43 ms. Além disso, ainda apresentou um melhoramento no número de travessias, obtendo o melhor desempenho no pior caso quando comparada a todas as outras estruturas. Este ganho de performance tem um custo direto associado à memória exigindo 75.02 MB para um milhão de elementos, sendo a estrutura mais pesada do nosso estudo. Assim, conclui-se que esta estrutura é ideal para cenários onde a travessia de ponteiros é o principal obstáculo.

Relativamente à segunda pergunta, a implementação da IARASS revelou ser um sucesso na diminuição do problema da dupla descida. Ao introduzir células skip com um apontador adicional, conseguimos reduzir o número de travessias no pior caso de 35 (variante Improved) para 27. Além disso, apresentou um tempo de execução de 65.00 ms, ultrapassando a Improved Myers List (77.46 ms) e a Advanced Myers List (401.43 ms). No que toca a uso de memória, a IARASS utiliza 50.86 MB, valor superior aos 38.15 MB da variante Improved e consumindo menos memória do que a Advanced Myers List (53.41 MB). Assim, conclui-se que esta estrutura apresenta um melhor desempenho, tanto a nível de velocidade de execução como também em número de travessias (afirmação apenas deve ser verdade para os dados que nós obtivemos), que a Advanced Myers List, conseguindo utilizar menos memória.

Em suma, a remoção da restrição permitiu-nos implementar duas novas estruturas de dados ... (TODO) A Dense Advanced Myers List é a melhor escolha quando a velocidade lookup é o mais importante e a memória não é um problema a ser considerado, continuando com a operação de cons. Por sua vez, a IARASS mostra ser uma estrutura de dados equilibrada que oferece melhorias consideráveis no desempenho de lookup com um aumento moderado na memória.

Referências

- [1] Eugene W. Myers. An applicative random-access stack. *Information Processing Letters*, 17(5):241–248, 1983.
- [2] Edward Peters, Yong Qi Foo, and Michael D. Adams. Pushing the information-theoretic limits of random access lists. 2025.